

UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION AND TECHNOLOGY



CSC10006 – Introduction to Database

PROJECT REPORT 1

Xv6 & Unix utilities

Student ID

23127140

23128307

Student Name

Dặng Nguyễn Duy Tuyên

Nguyễn Phạm Minh Thư

Lecturer in charge:

Lê Quốc Hòa

Ho Chi Minh City, 24 February 2025

Mục lục

1	Giới thiệu	4
1.1	Chủ đề	4
1.2	Mục đích	4
1.3	Thành tựu	4
1.4	Bảng phân công công việc	5
1.5	Instructors	5
2	pingpong.c	6
2.1	Ý tưởng và sơ đồ	6
2.2	Ý tưởng	6
2.3	Sơ đồ luồng thực thi	7
2.4	Cài đặt	7
2.4.1	Các thư viện cần thiết	7
2.4.2	Khai báo Pipe và Bộ đệm	8
2.4.3	Tạo tiến trình con bằng fork()	8
2.4.4	Tiến trình con đọc dữ liệu từ Pipe	8
2.4.5	Tiến trình con gửi dữ liệu lại	8
2.4.6	Tiến trình cha gửi dữ liệu trước	8
2.4.7	Tiến trình cha chờ tiến trình con	9
2.4.8	Tiến trình cha đọc dữ liệu từ Pipe	9
2.4.9	Kết thúc chương trình một cách sạch sẽ	9
2.5	Kết quả và bàn luận	9
2.5.1	Thứ tự thực thi chương trình	9
2.5.2	Kết quả	10
2.6	Thử thách và bài học rút ra	10
3	primes.c	11
3.1	Ý tưởng và sơ đồ	11
3.1.1	Ý tưởng	11
3.1.2	Sơ đồ	12
3.2	Cài đặt	13
3.2.1	Khai báo thư viện và hằng số	13
3.2.2	Khai báo biến	13
3.2.3	Hàm <code>primes</code>	13
3.2.4	Hàm <code>main</code>	14
3.2.5	Quản lý tài nguyên	14
3.3	Kết quả và bàn luận	15
3.3.1	Tóm tắt quá trình thực thi mã (<code>primes.c</code>)	15

3.3.2	Kết quả	15
3.4	Thử thách và bài học	16
4	find.c	17
4.1	Ý tưởng và sơ đồ	17
4.1.1	Ý tưởng	17
4.1.2	Sơ đồ	18
4.2	Cài đặt	19
4.2.1	Khai báo thư viện và hằng số	19
4.2.2	Khai báo biến	19
4.2.3	Hàm <code>find</code>	19
4.2.4	Hàm <code>main</code>	20
4.3	Kết quả và bàn luận	21
4.3.1	Tóm tắt quá trình thực thi mã (<code>find.c</code>)	21
4.3.2	Kết quả	21
4.4	Thử thách và bài học rút ra	22
4.4.1	Bài học rút ra	22
5	xargs.c	23
5.1	Ý tưởng và sơ đồ	23
5.2	Ý tưởng	23
5.3	Sơ đồ luồng thực thi	24
5.4	Cài đặt	25
5.5	Khai báo thư viện và hằng số	25
5.6	Khai báo biến	25
5.6.1	Lọc bỏ -n 1 khỏi danh sách tham số	25
5.6.2	Đọc dữ liệu từ <code>stdin</code>	25
5.6.3	Kiểm tra độ dài đối số	25
5.6.4	Phát hiện khoảng trắng hoặc xuống dòng	25
5.6.5	Tạo tiến trình con để thực thi lệnh	26
5.6.6	Chờ tiến trình con hoàn thành	26
5.7	Kết quả và bàn luận	26
5.7.1	Tóm tắt quá trình thực thi mã (<code>xargs.c</code>)	26
5.7.2	Khởi tạo chương trình	26
5.7.3	Đọc dữ liệu từ <code>stdin</code>	26
5.7.4	Thực thi lệnh	26
5.7.5	Xử lý từ cuối cùng	26
5.7.6	Đồng bộ hóa tiến trình	27
5.8	Kết quả	27
5.9	Thử thách và bài học rút ra	28

6	Kết quả và bàn luận	29
6.1	Kết quả	29
6.2	Kết luận	29
7	Tham khảo	29

Danh sách hình vẽ

1	Flowchart for ‘pingpong.c’ Process Communication	7
2	Kết quả chạy lệnh pingpong	10
3	Flowchart thể hiện thuật toán sử dụng trong primes.c	12
4	Kết quả chạy lệnh primes	15
5	Flowchart thể hiện thuật toán sử dụng trong find.c	18
6	Kết quả chạy lệnh find	21
7	Kết quả chạy lệnh echo hello too xargs echo bye	27
8	Kết quả chạy lệnh (echo 1 ; echo 2) xargs -n 1 echo	27
9	Kết quả chạy lệnh find . b xargs grep hello và sh < xargstest.sh . . .	28
10	Kết quả chạy lệnh make grade	29

1 Giới thiệu

xv6 là một hệ điều hành đơn giản, giống Unix được sử dụng trong các môi trường học thuật để dạy lập trình hệ thống cấp thấp. Dự án này giúp hiểu cách các hệ điều hành thực sự quản lý quy trình và xử lý đồng thời.

1.1 Chủ đề

Báo cáo này trình bày việc triển khai bốn tiện ích Unix pingpong, primes, find và xargs trong hệ điều hành xv6. Dự án liên quan đến việc sử dụng các lệnh gọi hệ thống như `fork()`, `pipe()`, `read()` và `write()` để đạt được giao tiếp giữa các tiến trình và thực thi đồng thời. Các kết quả đã được xác thực bằng các trường hợp thử nghiệm trong xv6 và các thách thức như đồng bộ hóa các tiến trình, tối ưu hóa bộ nhớ... đã được giải quyết. Báo cáo này cung cấp tổng quan về việc triển khai, những khó khăn gặp phải và bài học kinh nghiệm.

1.2 Mục đích

Mục đích của dự án này là triển khai một bộ tiện ích Unix: pingpong, primes, find và xargs trong hệ điều hành xv6. Các chương trình này trình bày các khái niệm chính của hệ điều hành như tạo quy trình, giao tiếp giữa các quy trình và lệnh gọi hệ thống.

1.3 Thành tựu

Trong quá trình thực hiện chương trình `xargs.c` trên hệ điều hành xv6, nhóm đã đạt được các thành tựu sau:

- **Hiểu sâu về cơ chế gọi hệ thống trong xv6:** Quá trình triển khai giúp nhóm nắm vững cách sử dụng `fork()`, `exec()`, `wait()` và `pipe()` trong lập trình hệ điều hành.
- **Giao tiếp giữa các tiến trình (IPC) hiệu quả:** Chương trình tận dụng pipe để truyền dữ liệu giữa tiến trình cha và tiến trình con một cách đồng bộ.
- **Xử lý đầu vào từ stdin linh hoạt:** Chương trình có thể đọc dữ liệu đầu vào, tách các đối số, và xử lý chúng một cách chính xác.
- **Đảm bảo đồng bộ hóa tiến trình:** Việc sử dụng `wait()` giúp chương trình hoạt động đúng thứ tự mong muốn, tránh lỗi race condition.
- **Tối ưu hóa bộ nhớ:** Giới hạn kích thước bộ đệm để tránh lỗi tràn bộ nhớ và đảm bảo chương trình hoạt động ổn định trong môi trường xv6.

1.4 Bảng phân công công việc

MSSV - Họ tên	Công việc	Mức độ hoàn thành
23127140 - Đặng Nguyễn Duy Tuyên	Cài đặt và viết báo cáo cho <code>find.c</code> và <code>primes.c</code>	100%
23127307 - Nguyễn Phạm Minh Thư	Cài đặt và viết báo cáo cho <code>pingpong.c</code> và <code>xargs.c</code>	100%

1.5 Instructors

- Cô Vũ Thị Mỹ Hằng (Lý thuyết)
- Thầy Lê Quốc Hòa (Thực hành)

Chúng em biết ơn các giảng viên hướng dẫn đã hỗ trợ. Kiến thức và phong cách giảng dạy của thầy, cô đã mang đến trải nghiệm học tập toàn diện giúp chúng em giải quyết dự án này.

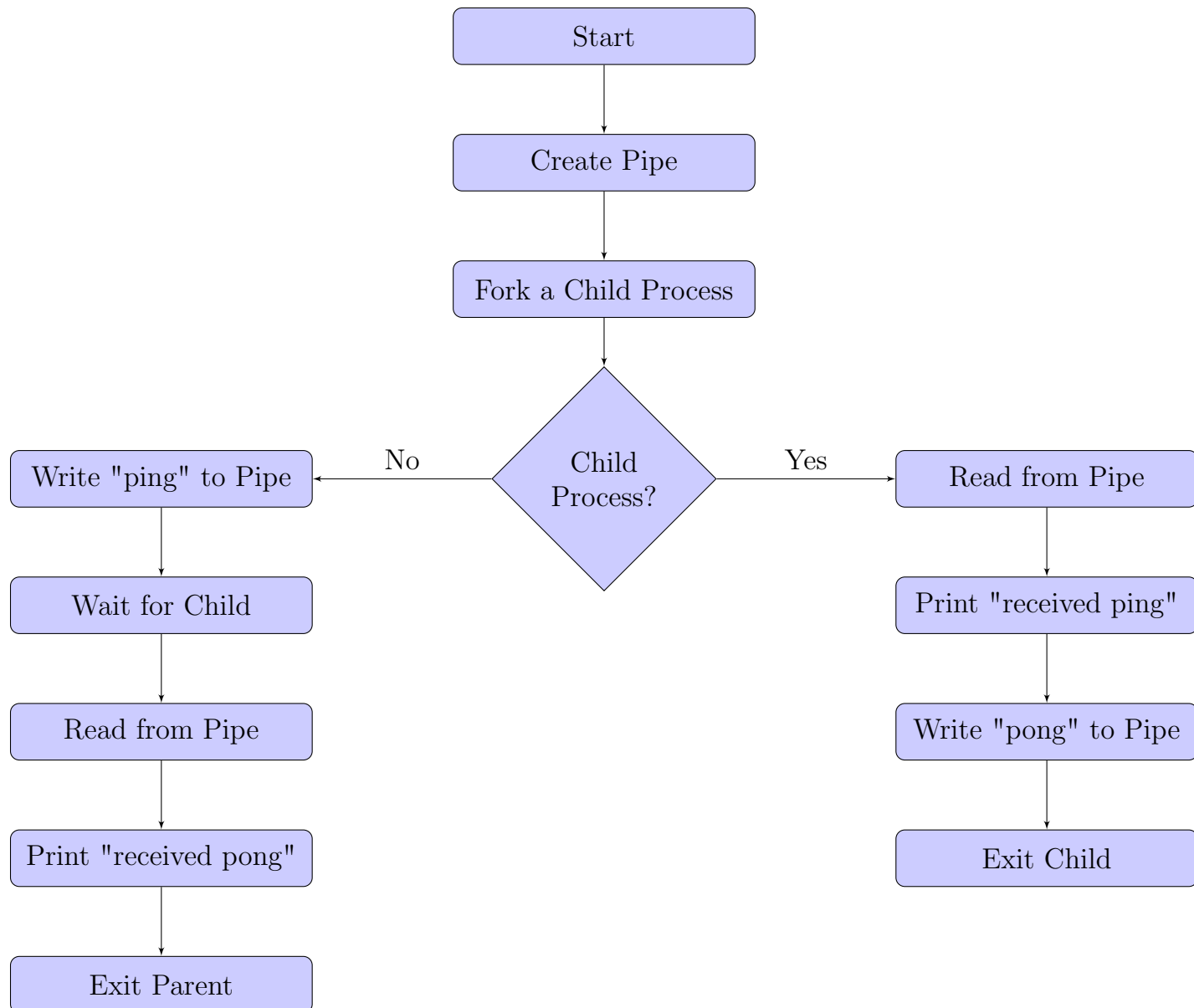
2 pingpong.c

2.1 Ý tưởng và sơ đồ

2.2 Ý tưởng

Chương trình `pingpong.c` minh họa sự giao tiếp giữa hai tiến trình trong hệ điều hành xv6 bằng cách sử dụng cơ chế **pipe**. Cụ thể, tiến trình cha gửi một byte dữ liệu đến tiến trình con, tiến trình con in ra thông báo nhận dữ liệu ("**received ping**") rồi gửi lại byte đó cho tiến trình cha và thoát. Tiến trình cha sau đó sẽ in ra thông báo nhận dữ liệu ("**received pong**") và thoát. Mục tiêu của chương trình này là: 1. Tạo pipe cho giao tiếp giữa các tiến trình. 2. Tạo ra một tiến trình con mới bằng cách sao chép tiến trình cha. 3. Gửi một byte từ tiến trình cha đến tiến trình con. 4. Tiến trình con in ra "đã nhận ping" và gửi byte trở lại. 5. Tiến trình cha đọc byte và in ra "đã nhận pong".

2.3 Sơ đồ luồng thực thi



Hình 1: Flowchart for 'pingpong.c' Process Communication

2.4 Cài đặt

2.4.1 Các thư viện cần thiết

- **types.h** và **stat.h**: Các thư viện tiêu chuẩn của xv6, định nghĩa các kiểu dữ liệu hệ thống.
- **user.h**: Chứa các định nghĩa hàm cho các lời gọi hệ thống như `fork()`, `pipe()`,

`write()`, `read()`, và `wait()`.

2.4.2 Khai báo Pipe và Bộ đệm

- Pipe được tạo bằng cách sử dụng `pipe(fd)`.
- `fd[0]` → Dùng để đọc dữ liệu.
- `fd[1]` → Dùng để ghi dữ liệu.
- `buf[1]` là bộ đệm một byte dùng để truyền dữ liệu qua pipe.

2.4.3 Tạo tiến trình con bằng `fork()`

- `fork()` tạo ra một tiến trình mới.
- Giá trị trả về xác định tiến trình cha hay tiến trình con:
 - Nếu `fork() == 0`, đây là tiến trình con.
 - Nếu `fork() > 0`, đây là tiến trình cha.

2.4.4 Tiến trình con đọc dữ liệu từ Pipe

- Tiến trình con đọc 1 byte từ `fd[0]` (đầu đọc của pipe).
- Nếu thành công, nó in ra:

```
<child_pid>: received ping
```

2.4.5 Tiến trình con gửi dữ liệu lại

- Tiến trình con ghi lại cùng một byte vào `fd[1]` (đầu ghi của pipe).
- Nếu `write()` thất bại, nó in ra thông báo lỗi.
- Lệnh `exit(0)`; rất quan trọng! Nếu thiếu nó, tiến trình con có thể tiếp tục chạy mãi ngoài ý muốn.

2.4.6 Tiến trình cha gửi dữ liệu trước

- Tiến trình cha ghi 1 byte vào pipe (`fd[1]`).
- Nếu thành công, tiến trình con có thể đọc byte này.

2.4.7 Tiến trình cha chờ tiến trình con

- Đảm bảo tiến trình con chạy trước (in ra "received ping" trước khi tiến trình cha đọc dữ liệu).
- Nếu không có `wait(0);`, tiến trình cha có thể thực thi trước, gây ra tình trạng race condition.

2.4.8 Tiến trình cha đọc dữ liệu từ Pipe

- Tiến trình cha đọc byte mà tiến trình con đã gửi lại.
- Nếu thành công, nó in ra:

```
<parent_pid>: received pong
```

2.4.9 Kết thúc chương trình một cách sạch sẽ

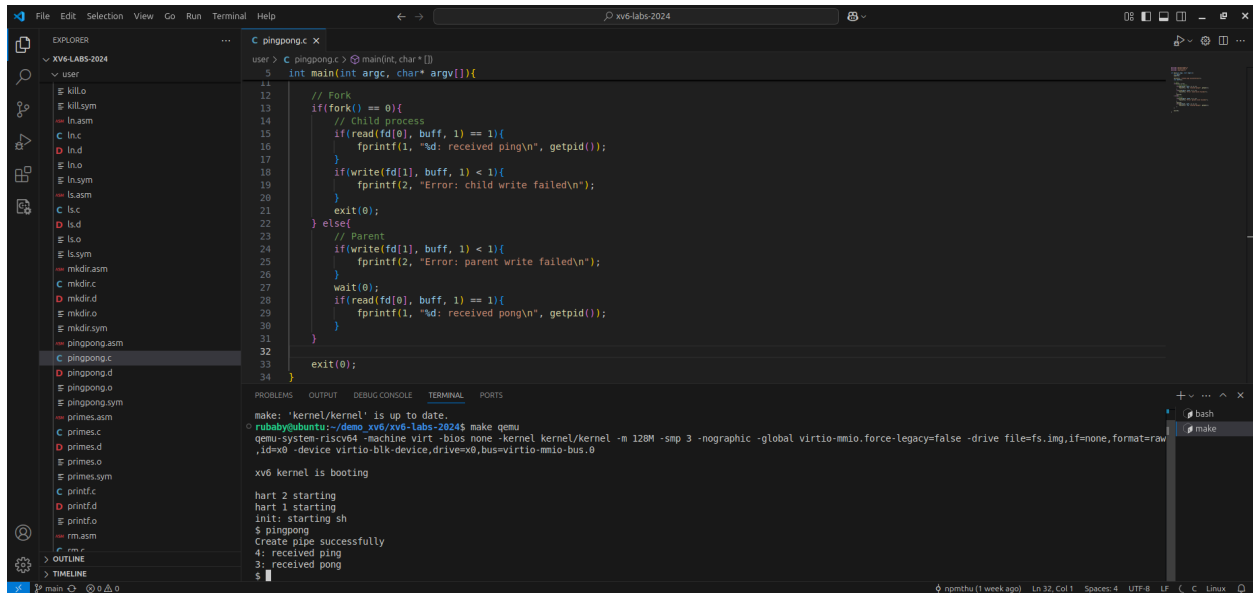
- Ngăn chặn việc thực thi ngoài ý muốn của bất kỳ đoạn mã còn sót lại.
- Đảm bảo tiến trình cha và tiến trình con kết thúc một cách sạch sẽ.

2.5 Kết quả và bàn luận

2.5.1 Thứ tự thực thi chương trình

1. Tiến trình cha tạo pipe.
2. Tiến trình cha gọi `fork()` để tạo tiến trình con.
3. Tiến trình con thực thi trước:
 - Đọc từ pipe (**ping**).
 - In ra "received ping".
 - Ghi lại (**pong**).
 - Thoát.
4. Tiến trình cha tiếp tục:
 - Ghi "ping" vào pipe.
 - Chờ tiến trình con kết thúc.
 - Đọc "pong" từ pipe.
 - In ra "received pong".
 - Thoát.

2.5.2 Kết quả



```
1  int main(int argc, char* argv[]){
2
3      // Fork
4      if(fork() == 0){
5          // Child process
6          if(read(fd[0], buff, 1) == 1){
7              fprintf(1, "%d: received ping\n", getpid());
8          }
9          if(write(fd[1], buff, 1) < 1){
10             fprintf(2, "Error: child write failed\n");
11         }
12         exit(0);
13     } else{
14         // Parent
15         if(write(fd[1], buff, 1) < 1){
16             fprintf(2, "Error: parent write failed\n");
17         }
18         wait(0);
19         if(read(fd[0], buff, 1) == 1){
20             fprintf(1, "%d: received pong\n", getpid());
21         }
22     }
23     exit(0);
24 }
```

```
make: 'kernel/kernel' is up to date.
root@ubuntu:~/demo_xv6/xv6-labs-2024# make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ pingpong
Create pipe successfully
4: received ping
3: received pong
$
```

Hình 2: Kết quả chạy lệnh pingpong

2.6 Thử thách và bài học rút ra

- **Đồng bộ hóa tiến trình:** Việc đảm bảo tiến trình cha đọc sau khi tiến trình con ghi là một vấn đề quan trọng. Lỗi ban đầu khi cài đặt là tiến trình con thoát ra ngay lập tức trước khi tiến trình cha có thể đọc nếu thiếu lệnh `exit(0)` trong tiến trình con và `wait(0)` trong tiến trình cha. Lệnh `exit(0)` bảo đảm việc tiến trình con thoát ra đúng cách và `wait(0)` bảo đảm việc tiến trình cha đợi tiến trình con thoát ra khi hoàn thành trước khi có thể đọc.
- **Sử dụng pipe đúng cách:** Không đóng file descriptor không cần thiết có thể gây lỗi chạy out of file descriptors.
- **Lỗi ghi dữ liệu:** Cần kiểm tra kỹ lỗi của hệ thống để đảm bảo tiến trình không bị lỗi do bộ nhớ đệm buffer.

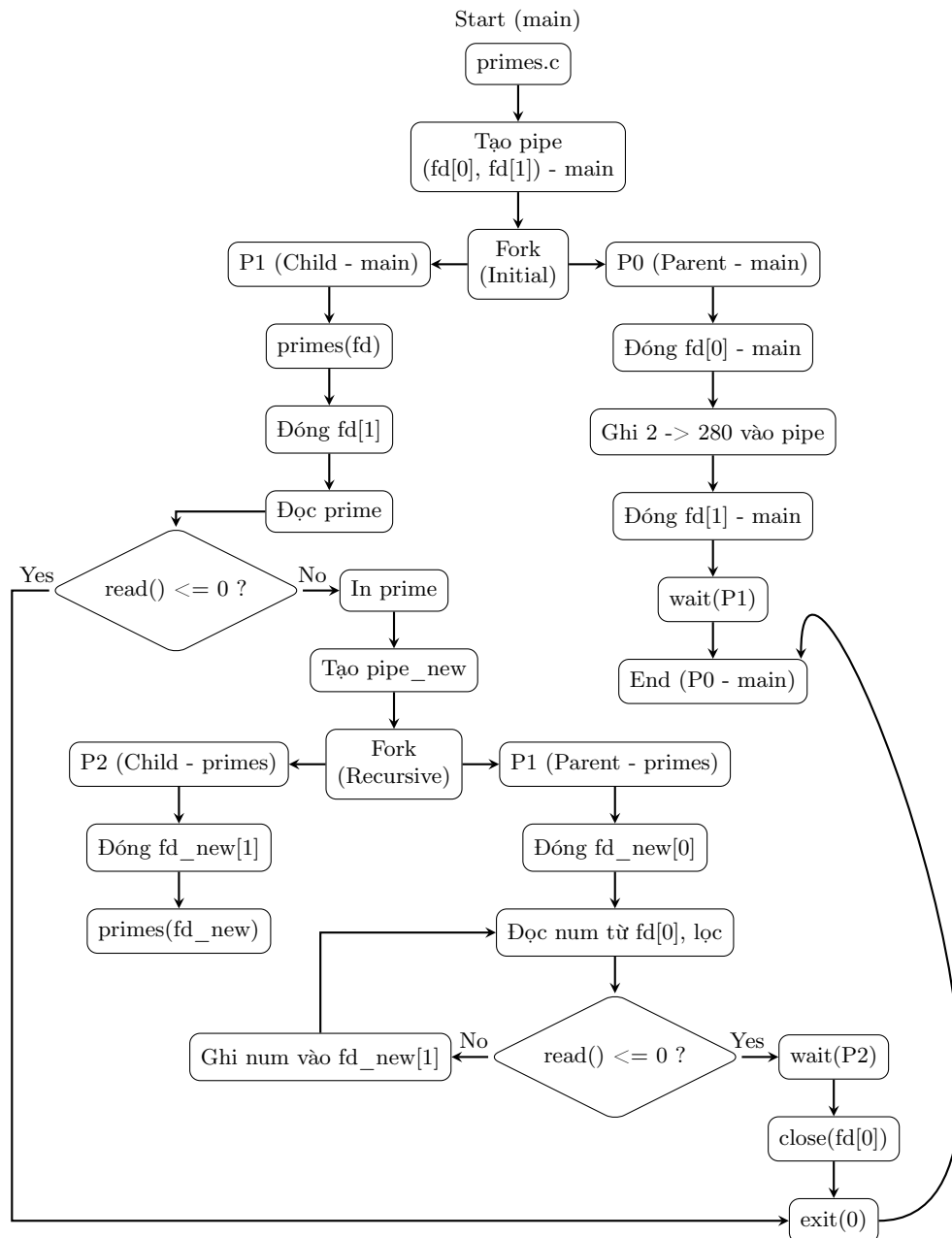
3 primes.c

3.1 Ý tưởng và sơ đồ

3.1.1 Ý tưởng

Ý tưởng chính của primes.c là sử dụng thuật toán Sieve of Eratosthenes để tìm các số nguyên tố. Ý tưởng chính là tạo một pipeline các tiến trình, mỗi tiến trình chịu trách nhiệm lọc ra các bội số của một số nguyên tố cụ thể.

3.1.2 Sơ đồ

Hình 3: Flowchart thể hiện thuật toán sử dụng trong `primes.c`

3.2 Cài đặt

3.2.1 Khai báo thư viện và hằng số

- `kernel/types.h`: Định nghĩa các kiểu dữ liệu cơ bản trong XV6.
- `user/user.h`: Chứa các hàm hệ thống như `fork()`, `pipe()`, `read()`, `write()`, và `wait()`.
- `stdio.h`: Định nghĩa các hàm input/output chuẩn trong c như `printf()` và `fprintf()`.

3.2.2 Khai báo biến

- `pipefd[2]`: Mảng chứa file descriptors của pipe hiện tại.
- `pipefd_next[2]`: Mảng chứa file descriptors của pipe tiếp theo.
- `prime`: Lưu số nguyên tố hiện tại.
- `num`: Lưu số đọc được từ pipe.
- `i`: Biến đếm để ghi các số từ 2 đến 280 vào pipe ban đầu.

3.2.3 Hàm primes

Hàm này thực hiện thuật toán Sieve of Eratosthenes bằng cách sử dụng đệ quy và pipeline:

- **Đọc số nguyên tố từ pipe:**
 - Đọc số đầu tiên từ pipe, số này được đảm bảo là số nguyên tố (vì hiển nhiên những số không phải là số nguyên tố đã được loại bỏ vì là bội của số nguyên tố trước).
 - Nếu không đọc được số nào (khi đó pipe đóng), kết thúc hàm.
- **In số nguyên tố:**
 - Số nguyên tố được in ra màn hình bằng `printf()`.
- **Tạo pipe mới và fork tiến trình con:**
 - Tạo một pipe mới để giao tiếp với tiến trình con.
 - Fork một tiến trình con để tiếp tục lọc các bội của số nguyên tố hiện tại.
- **Lọc các bội số:**
 - Tiến trình cha đọc các số từ pipe hiện tại, lọc ra các số không phải là bội số của số nguyên tố hiện tại, và ghi những số này vào pipe mới.
 - Đóng pipe và chờ tiến trình con kết thúc.

3.2.4 Hàm main

Hàm này khởi tạo pipeline và bắt đầu quá trình lọc số nguyên tố:

- **Tạo pipe ban đầu:**
 - Tạo một pipe để truyền các số từ 2 đến 280 vào pipeline.
- **Fork tiến trình con:**
 - Fork một tiến trình con để gọi hàm `primes` và bắt đầu quá trình lọc.
 - Tiến trình cha ghi các số từ 2 đến 280 vào pipe ban đầu.
- **Đóng pipe và chờ tiến trình con:**
 - Sau khi ghi xong, tiến trình cha đóng pipe và chờ tiến trình con kết thúc.

3.2.5 Quản lý tài nguyên

- **Đóng pipe:**
 - Các pipe được đóng đúng cách sau khi sử dụng để tránh rò rỉ tài nguyên.
 - Tiến trình con chỉ đọc từ pipe mới, trong khi tiến trình cha chỉ ghi vào pipe mới.
- **Đồng bộ hóa:**
 - Sử dụng `wait()` để đảm bảo tiến trình cha chờ tiến trình con hoàn thành trước khi kết thúc.

3.3 Kết quả và bàn luận

3.3.1 Tóm tắt quá trình thực thi mã (primes.c)

- Khởi tạo chương trình:
 - Tạo pipe ban đầu và fork tiến trình con để bắt đầu quá trình lọc số nguyên tố.
- Đọc và lọc số nguyên tố:
 - Mỗi tiến trình đọc số từ pipe, in số nguyên tố, và lọc các bội số của nó.
 - Các số không phải là bội số được truyền đến tiến trình tiếp theo trong pipeline.
- Kết thúc pipeline:
 - Quá trình tiếp tục cho đến khi không còn số nào được đọc từ pipe.
 - Các tiến trình đóng pipe và kết thúc một cách đồng bộ.

3.3.2 Kết quả

```

C primes.c x
user > C primes.c > primes(int [2])
4 // attribute__((noreturn)) -> Indicate that the function does not return
7 void primes(int pipefd[2]) __attribute__((noreturn));
8
9 // Implement sieve logic: Takes a pipe file descriptor array (pipefd) as an input
10 // The pipe is used to communicate with the previous process in the pipeline
11 void primes(int pipefd[2]) {
12     int prime; // Store the current prime number
13     int num; // Store numbers read from the pipe
14     int pipefd_next[2]; // File descriptors for the next pipe
15
16     close(pipefd[1]); // The process only reads from the pipe -> Doesn't need the write end -> Close th
17
18     // Read the first prime number from the pipe
19     // It is guaranteed to be prime since all multiples of previous primes have been filtered out
20     if (read(pipefd[0], &prime, sizeof(prime)) != sizeof(prime)) {
21         // ssize_t read(int fd, void *buf, size_t count);
22         // fd: File descriptor; buf: buffer to store read data; count: size in bytes of the buffer
23         // return: Number of bytes read; -1 on error; 0 on EOF
24
25         if (read(pipefd[0], &prime, sizeof(prime)) == 0) {
26             // If read returns 0 -> the pipe has been closed
27             close(pipefd[0]); // Close the read end
28             exit(0);
29         }
30         else {
31             // Actual error
32             fprintf(2, "primes: read error\n");
33             exit(1);
34         }
35     }
36
37     printf("prime %d\n", prime); // Print the prime number
38
39     pipe(pipefd_next); // Create a new pipe -> communicate with the next process in the pipeline
40
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
prime 37
prime 41
prime 43
prime 47
prime 53
prime 59
prime 61
prime 67
prime 71
prime 73
prime 79
prime 83
prime 89
prime 97
prime 101
prime 103
prime 107
prime 109
prime 113
prime 127
prime 131

```

Hình 4: Kết quả chạy lệnh primes

3.4 Thử thách và bài học

- **Thử thách:**

- Quản lý pipeline và các tiến trình con đòi hỏi sự cẩn thận để tránh resource leak, chẳng hạn như việc đóng các file descriptor không cần thiết.
- Debugging trong môi trường XV6 có thể khó khăn do thiếu công cụ hỗ trợ. Nên việc kiểm tra lỗi đọc/ghi pipe đòi hỏi phải được thực hiện cẩn thận.
- Đảm bảo rằng các tiến trình cha và con đồng bộ với nhau thông qua `wait()` để tránh tình trạng tiến trình cha kết thúc trước tiến trình con.

- **Bài học rút ra:**

- Hiểu rõ hơn về cách hoạt động của pipes và fork trong hệ điều hành.
- Nắm vững thuật toán Sieve of Eratosthenes và cách áp dụng nó trong lập trình hệ điều hành.
- Rèn luyện kỹ năng quản lý tài nguyên và đồng bộ hóa giữa các tiến trình.

4 `find.c`

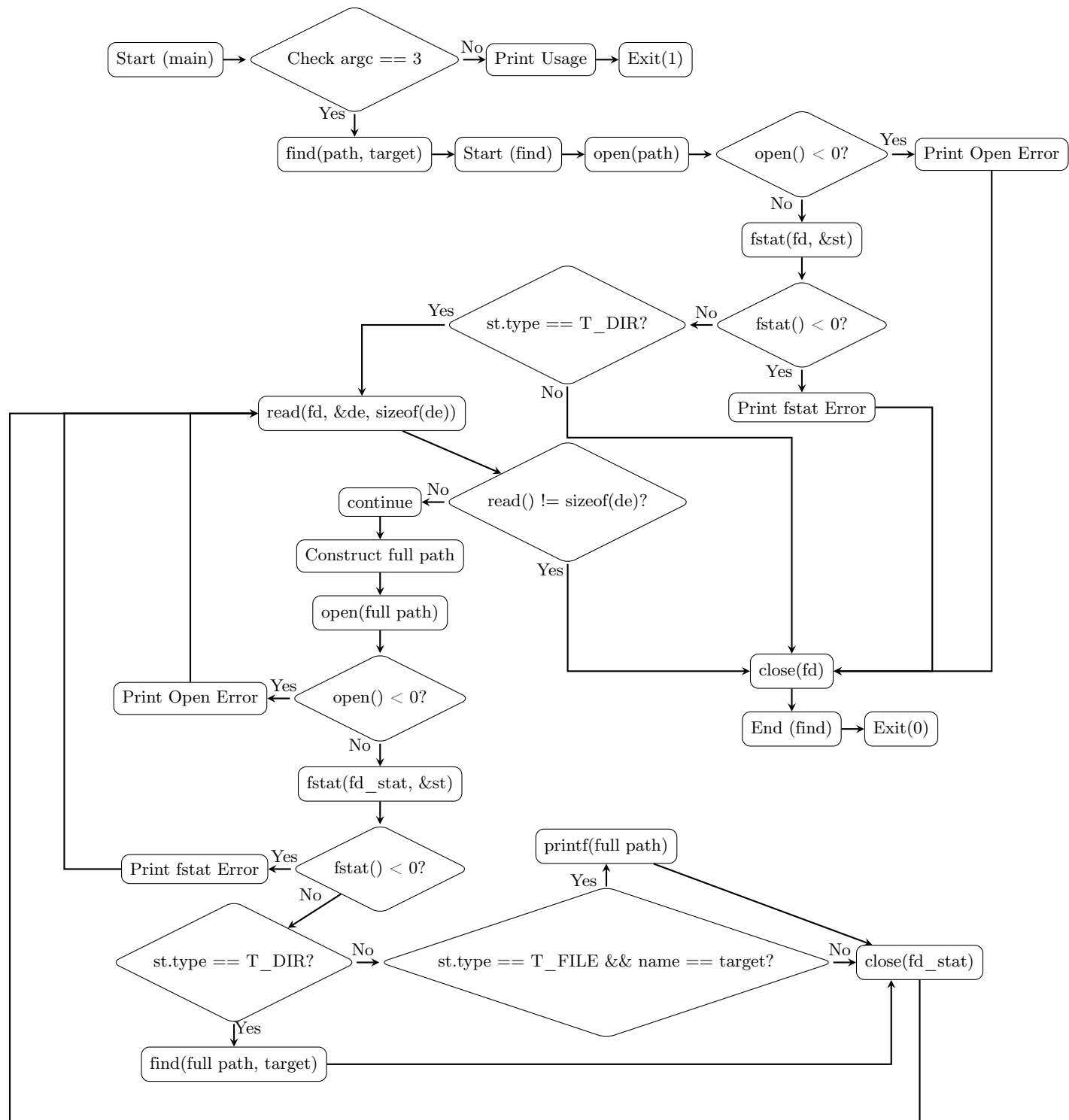
4.1 Ý tưởng và sơ đồ

4.1.1 Ý tưởng

`find.c` thực hiện tìm kiếm đệ quy các file trong thư mục và các thư mục con. Ý tưởng chính là:

- Mở thư mục được chỉ định và đọc từng entry (file hoặc thư mục con).
- Bỏ qua các entry trống hoặc thư mục đặc biệt (`.` và `..`).
- Xây dựng đường dẫn đầy đủ cho từng entry.
- Nếu entry là thư mục, gọi đệ quy để tìm kiếm trong thư mục con.
- Nếu entry là file và tên file khớp với tên đích, in ra đường dẫn đầy đủ của file.

4.1.2 Sơ đồ



Hình 5: Flowchart thể hiện thuật toán sử dụng trong find.c

4.2 Cài đặt

4.2.1 Khai báo thư viện và hằng số

- `kernel/types.h`: Định nghĩa các kiểu dữ liệu cơ bản trong XV6.
- `kernel/stat.h`: Định nghĩa cấu trúc `stat` để lưu trữ metadata của file (ví dụ: loại file, kích thước).
- `user/user.h`: Chứa các hàm hệ thống như `open()`, `read()`, `close()`, `stat()`, và `printf()`.
- `kernel/fs.h`: Định nghĩa cấu trúc thư mục (`dirent`) và các hằng số liên quan đến hệ thống file.

4.2.2 Khai báo biến

- `buf[512]`: Bộ đệm để lưu đường dẫn đầy đủ của file hoặc thư mục.
- `p`: Con trỏ để thao tác trên bộ đệm `buf`.
- `fd`: File descriptor để mở và đọc thư mục.
- `de`: Cấu trúc `dirent` để lưu thông tin về một entry trong thư mục (tên file và inode number).
- `st`: Cấu trúc `stat` để lưu metadata của file hoặc thư mục.

4.2.3 Hàm `find`

Hàm này thực hiện tìm kiếm đệ quy các file trong thư mục và các thư mục con:

- **Mở thư mục:**
 - Sử dụng `open()` để mở thư mục được chỉ định bởi `path`.
 - Nếu không thể mở, in thông báo lỗi và kết thúc hàm.
- **Lấy metadata của thư mục:**
 - Sử dụng `fstat()` để lấy metadata của thư mục.
 - Nếu không thể lấy metadata, in thông báo lỗi và đóng thư mục.
- **Kiểm tra loại thư mục:**
 - Kiểm tra xem `path` có phải là thư mục không bằng cách so sánh `st.type` với `T_DIR`.
 - Nếu không phải thư mục, in thông báo lỗi và đóng thư mục.

- **Đọc các entry trong thư mục:**

- Sử dụng `read()` để đọc từng entry trong thư mục.
- Bỏ qua các entry trống (`inode number = 0`) hoặc các thư mục đặc biệt (`.` và `..`).

- **Xây dựng đường dẫn đầy đủ:**

- Kết hợp `path` và tên entry để tạo đường dẫn đầy đủ.

- **Kiểm tra loại entry:**

- Nếu entry là thư mục (`T_DIR`), gọi đệ quy hàm `find` để tìm kiếm trong thư mục con.
- Nếu entry là file (`T_FILE`) và tên file khớp với `target`, in ra đường dẫn đầy đủ của file.

- **Đóng thư mục:**

- Đóng thư mục sau khi đọc xong các entry.

4.2.4 Hàm main

- **Kiểm tra số lượng đối số:**

- Nếu số lượng đối số không đúng (`argc != 3`), in thông báo lỗi và kết thúc chương trình.

- **Gọi hàm `find`:**

- Gọi hàm `find` với đối số là đường dẫn thư mục (`argv[1]`) và tên file cần tìm (`argv[2]`).

- **Kết thúc chương trình:**

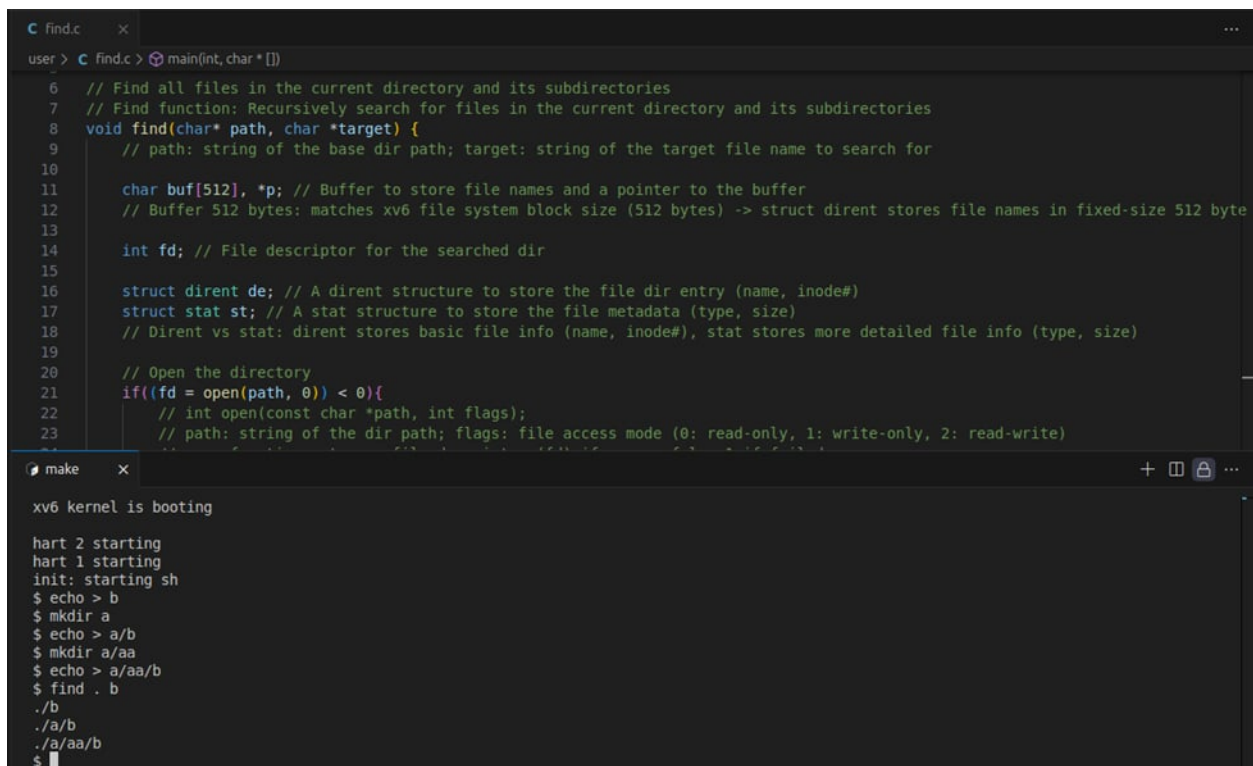
- Kết thúc chương trình với `exit(0)`.

4.3 Kết quả và bàn luận

4.3.1 Tóm tắt quá trình thực thi mã (find.c)

- Khởi tạo chương trình:
 - Kiểm tra số lượng đối số và gọi hàm `find` để bắt đầu tìm kiếm.
- Tìm kiếm đệ quy:
 - Hàm `find` duyệt qua từng entry trong thư mục và các thư mục con.
 - Nếu tìm thấy file khớp với tên đích, in ra đường dẫn đầy đủ của file.
- Kết thúc chương trình:
 - Đóng các thư mục và kết thúc chương trình một cách an toàn.

4.3.2 Kết quả



```
user > C find.c > main(int, char* [])
6 // Find all files in the current directory and its subdirectories
7 // Find function: Recursively search for files in the current directory and its subdirectories
8 void find(char* path, char *target) {
9     // path: string of the base dir path; target: string of the target file name to search for
10
11     char buf[512], *p; // Buffer to store file names and a pointer to the buffer
12     // Buffer 512 bytes: matches xv6 file system block size (512 bytes) -> struct dirent stores file names in fixed-size 512 byte
13
14     int fd; // File descriptor for the searched dir
15
16     struct dirent de; // A dirent structure to store the file dir entry (name, inode#)
17     struct stat st; // A stat structure to store the file metadata (type, size)
18     // Dirent vs stat: dirent stores basic file info (name, inode#), stat stores more detailed file info (type, size)
19
20     // Open the directory
21     if((fd = open(path, 0)) < 0){
22         // int open(const char *path, int flags);
23         // path: string of the dir path; flags: file access mode (0: read-only, 1: write-only, 2: read-write)
24     }
25
26     // Print the directory contents
27     if(fd > 0){
28         struct dirent* d;
29         while((d = readdir(fd)) != NULL){
30             // Skip "." and ".."
31             if(d->name[0] == '.' || d->name[0] == '\0') continue;
32             // Recursively search for the target file
33             find(d->name, target);
34         }
35     }
36 }
37
38 int main(int argc, char** argv){
39     if(argc != 2){
40         printf("Usage: find <target>\n");
41         return 1;
42     }
43     find(".", argv[1]);
44     return 0;
45 }
```

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
$
```

Hình 6: Kết quả chạy lệnh find

4.4 Thử thách và bài học rút ra

- **Quản lý bộ đệm:**

- Việc xây dựng path đầy đủ bằng cách kết hợp string đòi hỏi phải quản lý bộ đệm buf [512] cẩn thận.

- **Xử lý lỗi:**

- Cần kiểm tra kết quả trả về của các hàm hệ thống như `open()`, `read()`, và `fstat()` để xử lý lỗi kịp thời.

4.4.1 Bài học rút ra

- **Quản lý tài nguyên:**

- Luôn đóng file descriptor sau khi sử dụng để tránh rò rỉ tài nguyên.

- **Kiểm tra lỗi:**

- Kiểm tra kết quả return của các hàm hệ thống giúp phát hiện và xử lý lỗi kịp thời, đảm bảo chương trình hoạt động ổn định.

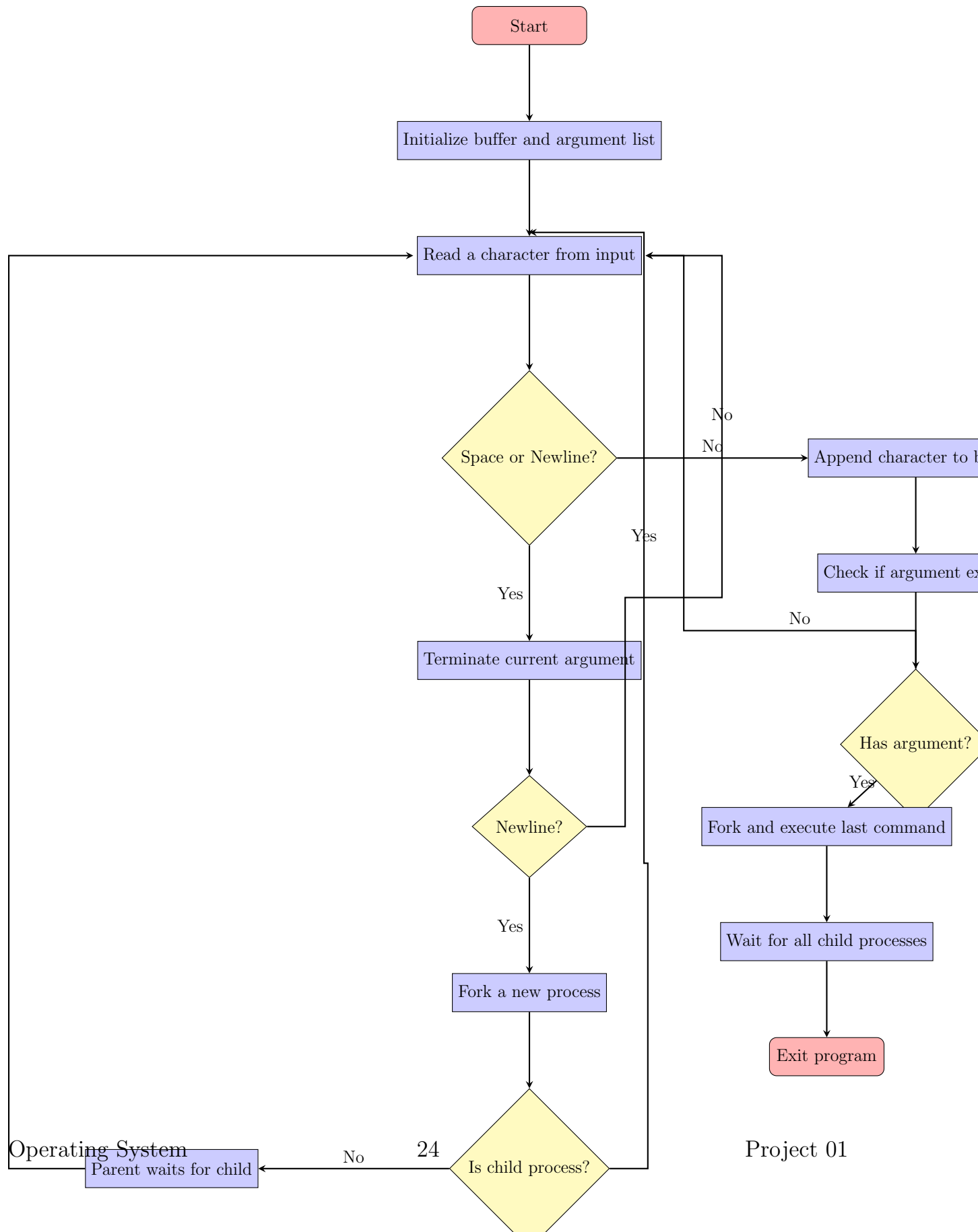
5 `xargs.c`

5.1 Ý tưởng và sơ đồ

5.2 Ý tưởng

Chương trình `xargs.c` trong xv6 mô phỏng lệnh **xargs** của UNIX, dùng để lấy dữ liệu từ **stdin** và thêm nó làm đối số cho một lệnh khác. Chương trình này giúp hiểu cách thức **exec()** thực thi tiến trình mới với tham số từ dòng nhập.

5.3 Sơ đồ luồng thực thi



5.4 Cài đặt

5.5 Khai báo thư viện và hằng số

- **types.h**: Định nghĩa kiểu dữ liệu cơ bản trong xv6.
- **user.h**: Chứa các hàm hệ thống như `fork()`, `exec()`, `wait()`, `read()`, `write()`.
- **param.h**: Định nghĩa `MAXARG`, số lượng tham số tối đa cho `exec()`.
- **BUFSIZE**: Kích thước bộ đệm lưu dữ liệu từ `stdin`.

5.6 Khai báo biến

- `buf[BUFSIZE]`: Bộ đệm lưu dữ liệu nhập từ `stdin`.
- `xargv[MAXARG]`: Mảng chứa danh sách tham số truyền cho `exec()`.
- `args, arge`: Trỏ đến vị trí đầu và cuối của một đối số trong `buf`.
- `ch`: Ký tự đọc từ `stdin`.
- `readbytes`: Số byte đọc được từ `stdin`.
- `hasarg`: Đánh dấu có đối số trong `buf` hay không.
- `childpid`: PID của tiến trình con sau khi `fork()`.

5.6.1 Loại bỏ -n 1 khỏi danh sách tham số

Nếu người dùng nhập -n 1, chương trình bỏ qua tùy chọn này. Các tham số còn lại của lệnh sẽ được lưu vào `xargv`.

5.6.2 Đọc dữ liệu từ `stdin`

`read(0, ch, sizeof(char))`: Đọc từng ký tự từ `stdin`. `args, arge` khởi tạo để trỏ vào `buf`, giúp tách từng đối số.

5.6.3 Kiểm tra độ dài đối số

Nếu `buf` đầy, chương trình in lỗi "argument too long" và đặt lại `buf`.

5.6.4 Phát hiện khoảng trắng hoặc xuống dòng

Khi gặp khoảng trắng (' ') hoặc xuống dòng ('\n'): Kết thúc một đối số bằng ' ' (null-terminate). Nếu `hasarg = 1`, tức là `buf` đang chứa một đối số hợp lệ.

5.6.5 Tạo tiến trình con để thực thi lệnh

`fork()`: Tạo tiến trình con để thực thi lệnh. Tiến trình con gọi `exec(xargv[0], xargv)`, thay thế tiến trình hiện tại bằng lệnh. Nếu `exec()` thất bại, chương trình in lỗi "exec failed".

5.6.6 Chờ tiến trình con hoàn thành

`wait(0)`: Đảm bảo tiến trình con hoàn tất trước khi thực hiện lệnh tiếp theo. Đặt lại buf để tiếp tục đọc đối số mới. Xử lý trường hợp dòng nhập không kết thúc bằng newline. Nếu `stdin` không kết thúc bằng , lệnh cuối cùng vẫn được thực thi.

5.7 Kết quả và bàn luận

5.7.1 Tóm tắt quá trình thực thi mã (`xargs.c`)

5.7.2 Khởi tạo chương trình

- Đọc các tham số dòng lệnh (bỏ qua tùy chọn `-n 1` nếu có).
- Lưu chúng vào mảng `x_argv`, sẽ được sử dụng để thực thi lệnh.
- Khởi tạo bộ đệm buf [`BUF_SIZE`] để lưu dữ liệu từ `stdin`.

5.7.3 Đọc dữ liệu từ `stdin`

- Đọc từng ký tự một từ `stdin`.
- Xác định ranh giới từ (dấu cách hoặc xuống dòng).
- Lưu các từ vào buf và đánh dấu chúng là các đối số mới.

5.7.4 Thực thi lệnh

- Khi gặp dấu cách hoặc ký tự xuống dòng:
 - Từ đang lưu trong bộ đệm sẽ được kết thúc bằng ký tự `\0`.
 - Thêm từ đó vào danh sách tham số `x_argv`.
- Tạo một tiến trình con bằng `fork()`.
 - **Tiến trình con** gọi `exec()` để thực thi lệnh với các tham số đã thu thập.
 - **Tiến trình cha** gọi `wait()` để chờ tiến trình con hoàn tất.

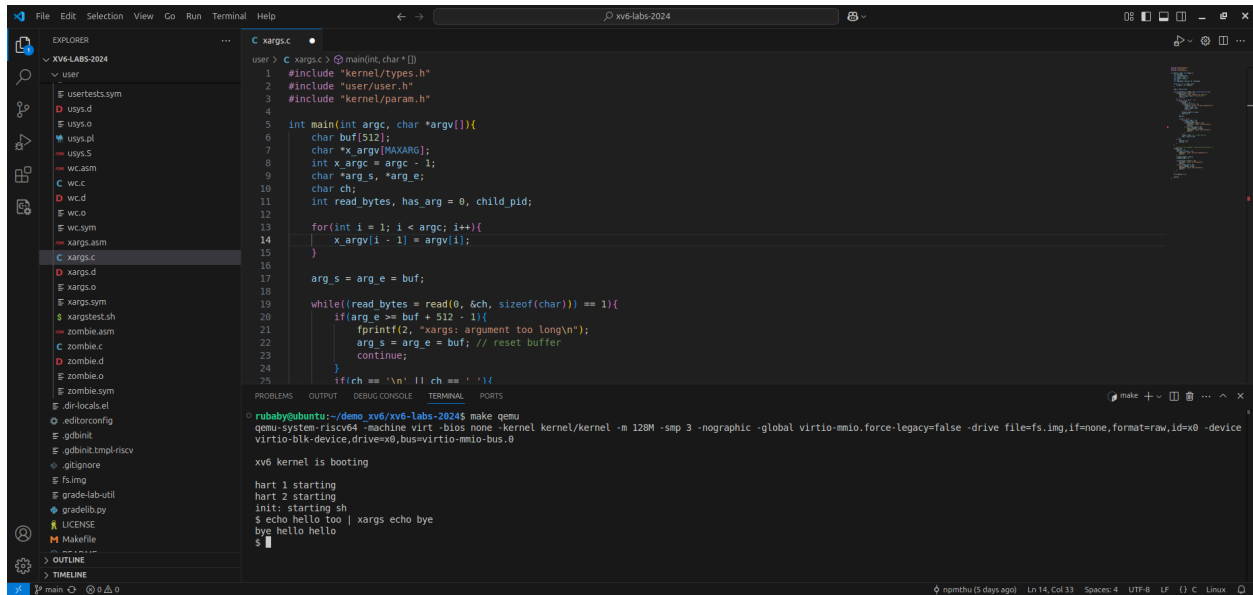
5.7.5 Xử lý từ cuối cùng

- Nếu đầu vào không kết thúc bằng ký tự xuống dòng (`\n`), chương trình vẫn thực thi lệnh với từ cuối cùng.

5.7.6 Đồng bộ hóa tiến trình

- Tiến trình cha gọi `wait(0)` để đảm bảo rằng mỗi tiến trình con kết thúc trước khi tiếp tục với tiến trình tiếp theo.

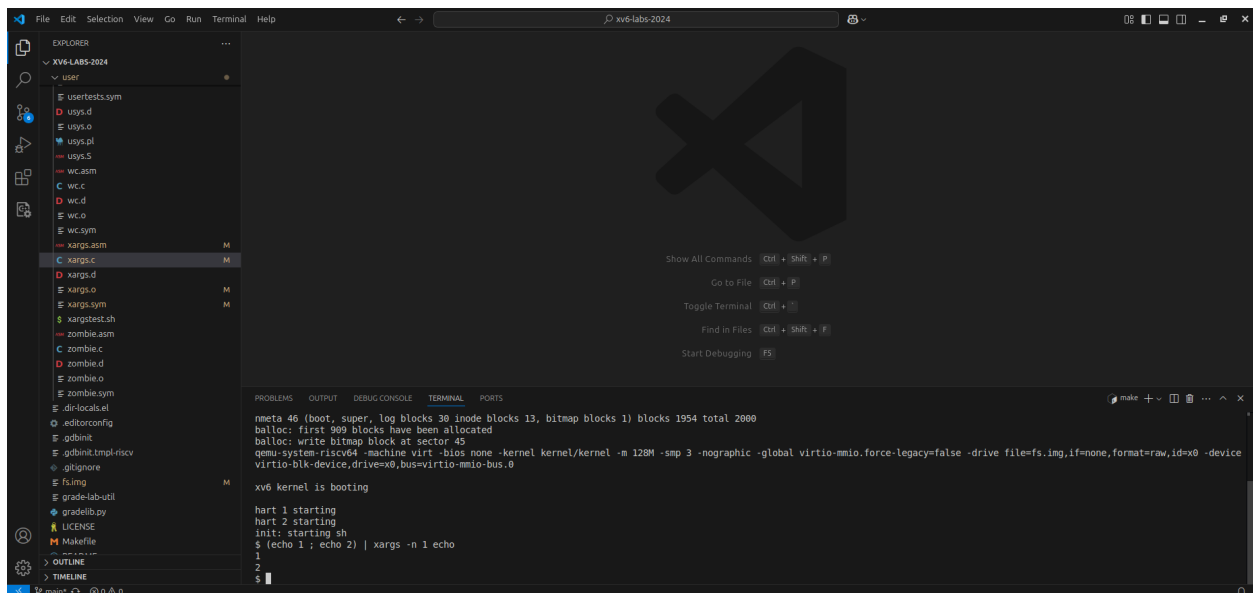
5.8 Kết quả



```
user > C xargs.c > @ main(int, char*[])
1 #include "kernel/types.h"
2 #include "user/user.h"
3 #include "kernel/param.h"
4
5 int main(int argc, char *argv[]){
6     char buf[512];
7     char *x_argv[MAXARG];
8     int x_argc = argc - 1;
9     char *arg_s, *arg_e;
10    char ch;
11    int read_bytes, has_arg = 0, child_pid;
12
13    for(int i = 1; i < argc; i++){
14        x_argv[i - 1] = argv[i];
15    }
16
17    arg_s = arg_e = buf;
18
19    while((read_bytes = read(0, &ch, sizeof(char))) == 1){
20        if(arg_e == buf + 512 - 1){
21            printf(2, "xargs: argument too long\n");
22            arg_s = arg_e = buf; // reset buffer
23            continue;
24        }
25        if(ch == '\n' || ch == ' '){
26            if(arg_e - arg_s > 0){
27                child_pid = fork1();
28                if(child_pid == 0){
29                    execl(arg_s, "sh", "-c", "echo %s", arg_s, arg_e);
30                    _exit(1);
31                }
32                waitpid(child_pid, 0, 0);
33            }
34            arg_s = arg_e = buf;
35        }
36    }
37    if(arg_e - arg_s > 0){
38        child_pid = fork1();
39        if(child_pid == 0){
40            execl(arg_s, "sh", "-c", "echo %s", arg_s, arg_e);
41            _exit(1);
42        }
43        waitpid(child_pid, 0, 0);
44    }
45}
```

```
rubab@ubuntu:~/demo_xv6/xv6-labs-2024$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ echo hello too | xargs echo bye
bye hello hello
$
```

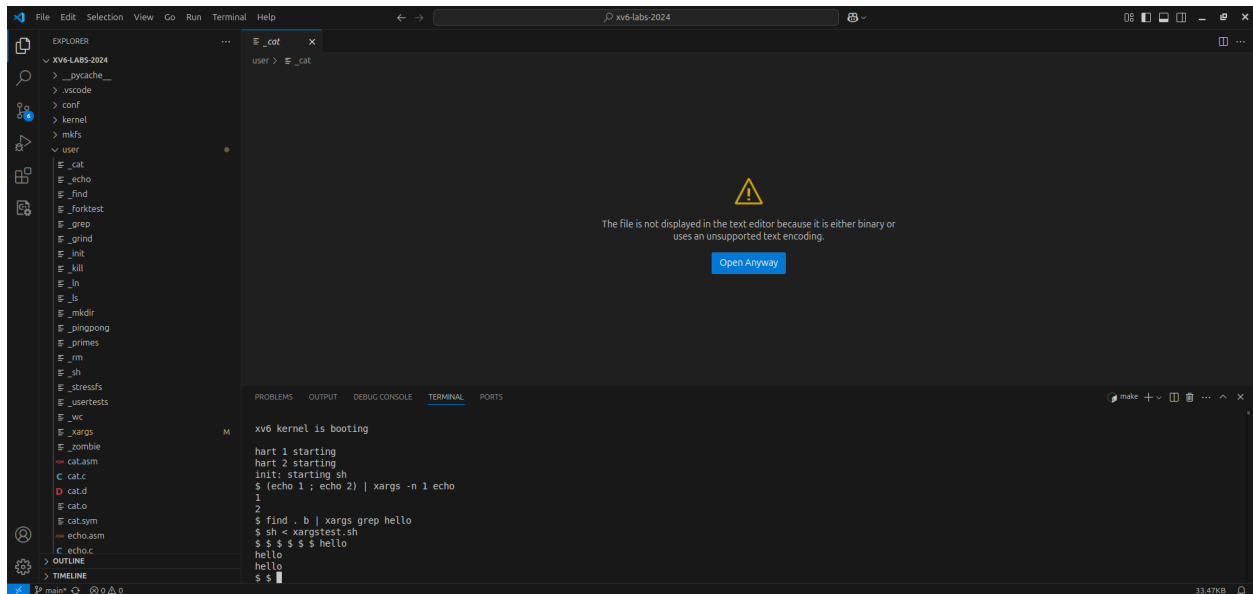
Hình 7: Kết quả chạy lệnh `echo hello too | xargs echo bye`



```
user > C xargs.c > @ main(int, char*[])
1 #include "kernel/types.h"
2 #include "user/user.h"
3 #include "kernel/param.h"
4
5 int main(int argc, char *argv[]){
6     char buf[512];
7     char *x_argv[MAXARG];
8     int x_argc = argc - 1;
9     char *arg_s, *arg_e;
10    char ch;
11    int read_bytes, has_arg = 0, child_pid;
12
13    for(int i = 1; i < argc; i++){
14        x_argv[i - 1] = argv[i];
15    }
16
17    arg_s = arg_e = buf;
18
19    while((read_bytes = read(0, &ch, sizeof(char))) == 1){
20        if(arg_e == buf + 512 - 1){
21            printf(2, "xargs: argument too long\n");
22            arg_s = arg_e = buf; // reset buffer
23            continue;
24        }
25        if(ch == '\n' || ch == ' '){
26            if(arg_e - arg_s > 0){
27                child_pid = fork1();
28                if(child_pid == 0){
29                    execl(arg_s, "sh", "-c", "echo %s", arg_s, arg_e);
30                    _exit(1);
31                }
32                waitpid(child_pid, 0, 0);
33            }
34            arg_s = arg_e = buf;
35        }
36    }
37    if(arg_e - arg_s > 0){
38        child_pid = fork1();
39        if(child_pid == 0){
40            execl(arg_s, "sh", "-c", "echo %s", arg_s, arg_e);
41            _exit(1);
42        }
43        waitpid(child_pid, 0, 0);
44    }
45}
```

```
rubab@ubuntu:~/demo_xv6/xv6-labs-2024$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ (echo 1 ; echo 2) | xargs -n 1 echo
1
2
$
```

Hình 8: Kết quả chạy lệnh `(echo 1 ; echo 2) | xargs -n 1 echo`



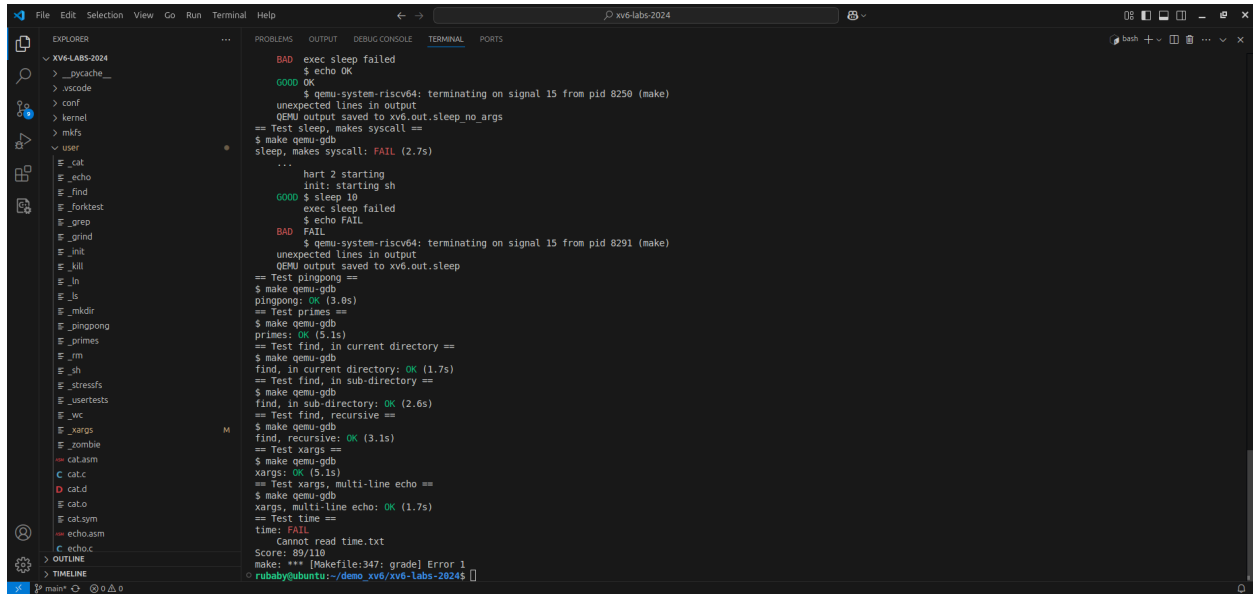
Hình 9: Kết quả chạy lệnh `find . b | xargs grep hello` và `sh < xargstest.sh`

5.9 Thử thách và bài học rút ra

- **Xử lý stdin:** Đọc và tách dữ liệu đúng cách tránh lỗi ghi bộ nhớ. Dữ liệu có thể có dấu cách (' '), xuống dòng (") → Cần tách từng đối số chính xác. Phải đảm bảo bộ nhớ không bị tràn (buffer overflow) khi đọc dữ liệu đầu vào.
- **Quản lý tiến trình:** Sử dụng `fork()` và `exec()` để gọi lệnh an toàn. `fork()` tạo tiến trình con để thực thi lệnh. `exec()` thực thi chương trình mới, nếu lỗi cần báo rõ `exec failed (command: <tên lệnh>)`. `wait()` giúp tiến trình cha đợi tiến trình con kết thúc để tránh zombie process.
- **Xử lý đúng cú pháp dòng lệnh:** Giới hạn bộ nhớ đệm để tránh lỗi `argument too long`.
- **Hạn chế bộ nhớ:** `-n 1` đảm bảo mỗi lần thực thi chỉ nhận một tham số duy nhất. Trong xv6, ta cần xử lý từng đối số một, mà không cần tối ưu hóa việc gom nhóm nhiều đối số như Unix. Cách khắc phục: Bỏ qua `-n 1` trong danh sách đối số để tránh lỗi này.
- **Viết code đúng nguyên tắc UNIX:** Làm đúng một việc và làm tốt việc đó: Chỉ thực hiện `xargs -n 1`, không tối ưu hóa quá mức. Đơn giản, dễ hiểu: Dùng vòng lặp để đọc đầu vào, tách từng tham số, và thực thi ngay lập tức. Kiểm tra lỗi cẩn thận: Xử lý lỗi `fork()`, `exec()`, và giới hạn tham số.

6 Kết quả và bàn luận

6.1 Kết quả



```

BAD  exec sleep failed
$ echo OK
GOOD OK
$ qemu-system-riscv64: terminating on signal 15 from pid 8250 (make)
unexpected lines in output
QEMU output saved to xv6.out.sleep_no_args
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: FAIL (2.7s)
...
hart 2 starting
init: starting sh
GOOD $ sleep 10
exec sleep failed
$ echo FAIL
BAD FAIL
$ qemu-system-riscv64: terminating on signal 15 from pid 8291 (make)
unexpected lines in output
QEMU output saved to xv6.out.sleep
== Test pingpong ==
$ make qemu-gdb
pingpong: OK (3.0s)
== Test primes ==
$ make qemu-gdb
primes: OK (5.1s)
== Test find, in current directory ==
$ make qemu-gdb
find, in current directory: OK (1.7s)
== Test find, in sub-directory ==
$ make qemu-gdb
find, in sub-directory: OK (2.6s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (3.1s)
== Test xargs ==
$ make qemu-gdb
xargs: OK (5.1s)
== Test xargs, multi-line echo ==
$ make qemu-gdb
xargs, multi-line echo: OK (1.7s)
== Test time ==
time: FAIL
Cannot read time.txt
Score: 89/119
make: *** [Makefile:347: grade] Error 1
rubaby@ubuntu:~/demo_xv6/xv6-labs-2024$

```

Hình 10: Kết quả chạy lệnh make grade

Về kết quả chạy kiểm thử các chương trình trong xv6-labs, với kết quả như sau:
Thành công: pingpong, primes, find (các trường hợp), xargs (các trường hợp).

6.2 Kết luận

Dự án này giúp nhóm có cơ hội thực hành và áp dụng các kiến thức về hệ điều hành, đặc biệt là:

- Hiểu rõ cách hệ điều hành quản lý tiến trình và giao tiếp giữa các tiến trình.
- Cải thiện kỹ năng lập trình hệ thống với các lời gọi hệ thống quan trọng.
- Giải quyết các vấn đề thực tế liên quan đến lập trình hệ điều hành, chẳng hạn như xử lý lỗi khi đọc dữ liệu, quản lý bộ nhớ và tối ưu hóa hiệu suất.

Chương trình `xargs.c` đã được kiểm thử với nhiều đầu vào khác nhau như trong yêu cầu để đảm bảo tính chính xác và ổn định. Trong tương lai, chương trình có thể được mở rộng để hỗ trợ nhiều đối số hơn và cải thiện hiệu suất khi xử lý dữ liệu lớn.

7 Tham khảo

- [Lab: Xv6 and Unix utilities](#)

- [xv6 Kernel](#)
- [ChatGPT](#)